```c
//#############################################################################
// FILE:   HWstarter_main.c
//
// TITLE:  HW Starter
//#############################################################################

// Included Files
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>
#include <math.h>
#include <limits.h>
#include "F28x_Project.h"
#include "driverlib.h"
#include "device.h"
#include "F28379dSerial.h"
#include "LEDPatterns.h"
#include "song.h"
#include "dsp.h"
#include "fpu32/fpu_rfft.h"

#define PI          3.14159265358979323846264338327795
#define TWOPI       6.28318530717958647692528676655900
#define HALFPI      1.57079632679489661923132169163980
// The Launchpad's CPU Frequency set to 200 you should not change this value
#define LAUNCHPAD_CPU_FREQUENCY 200


// Interrupt Service Routines predefinition
__interrupt void cpu_timer0_isr(void);
__interrupt void cpu_timer1_isr(void);
__interrupt void cpu_timer2_isr(void);
__interrupt void SWI_isr(void);

// Count variables
uint32_t numTimer0calls = 0;
uint32_t numSWIcalls = 0;
extern uint32_t numRXA;
uint16_t UARTPrint = 0;
uint16_t LEDdisplaynum = 0;


void main(void)
{
    // PLL, WatchDog, enable Peripheral Clocks
    // This example function is found in the F2837xD_SysCtrl.c file.
    InitSysCtrl();

    InitGpio();

    // Blue LED on LaunchPad
    GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO31 = 1;

    // Red LED on LaunchPad
    GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBSET.bit.GPIO34 = 1;

    // LED1 and PWM Pin
    GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 5);
    GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);

    // LED2
    GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;

    // LED3
    GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;

    // LED4
    GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;

    // LED5
    GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;

    // LED6
```

```
    GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;

    // LED7
    GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;

    // LED8
    GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;

    // LED9
    GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;

    // LED10
    GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;

    // LED11
    GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;

    // LED12
    GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;

    // LED13
    GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(157, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO157 = 1;

    // LED14
    GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(158, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO158 = 1;

    // LED15
    GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(159, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO159 = 1;

    // LED16
    GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(160, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPFCLEAR.bit.GPIO160 = 1;

    //WIZNET Reset
    GPIO_SetupPinMux(0, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(0, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO0 = 1;

    //ESP8266 Reset
    GPIO_SetupPinMux(1, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(1, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO1 = 1;

    //SPIRAM  CS  Chip Select
    GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO19 = 1;

    //DRV8874 #1 DIR  Direction
    GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO29 = 1;

    //DRV8874 #2 DIR  Direction
    GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBSET.bit.GPIO32 = 1;

    //DAN28027  CS  Chip Select
    GPIO_SetupPinMux(9, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(9, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO9 = 1;

    //MPU9250  CS  Chip Select
    GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPCSET.bit.GPIO66 = 1;

```

```
    //WIZNET  CS  Chip Select
    GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPDSET.bit.GPIO125 = 1;

    //PushButton 1
    GPIO_SetupPinMux(4, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(4, GPIO_INPUT, GPIO_PULLUP);

    //PushButton 2
    GPIO_SetupPinMux(5, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(5, GPIO_INPUT, GPIO_PULLUP);

    //PushButton 3
    GPIO_SetupPinMux(6, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(6, GPIO_INPUT, GPIO_PULLUP);

    //PushButton 4
    GPIO_SetupPinMux(7, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(7, GPIO_INPUT, GPIO_PULLUP);

    //Joy Stick Pushbutton
    GPIO_SetupPinMux(8, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(8, GPIO_INPUT, GPIO_PULLUP);


    // Clear all interrupts and initialize PIE vector table:
    // Disable CPU interrupts
    DINT;

    // Initialize the PIE control registers to their default state.
    // The default state is all PIE interrupts disabled and flags
    // are cleared.
    // This function is found in the F2837xD_PieCtrl.c file.
    InitPieCtrl();

    // Disable CPU interrupts and clear all CPU interrupt flags:
    IER = 0x0000;
    IFR = 0x0000;

    // Initialize the PIE vector table with pointers to the shell Interrupt
    // Service Routines (ISR).
    // This will populate the entire table, even if the interrupt
    // is not used in this example.  This is useful for debug purposes.
    // The shell ISR routines are found in F2837xD_DefaultIsr.c.
    // This function is found in F2837xD_PieVect.c.
    InitPieVectTable();

    // Interrupts that are used in this example are re-mapped to
    // ISR functions found within this project
    EALLOW;  // This is needed to write to EALLOW protected registers
    PieVectTable.TIMER0_INT = &cpu_timer0_isr;
    PieVectTable.TIMER1_INT = &cpu_timer1_isr;
    PieVectTable.TIMER2_INT = &cpu_timer2_isr;
    PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
    PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
    PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
    PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
    PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
    PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
    PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
    PieVectTable.SCID_TX_INT = &TXDINT_data_sent;

    PieVectTable.EMIF_ERROR_INT = &SWI_isr;
    EDIS;    // This is needed to disable write to EALLOW protected registers


    // Initialize the CpuTimers Device Peripheral. This function can be
    // found in F2837xD_CpuTimers.c
    InitCpuTimers();

    // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
    // 200MHz CPU Freq,                     Period (in uSeconds)
    ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);
    ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 20000);
    ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000);

    // Enable CpuTimer Interrupt bit TIE
    CpuTimer0Regs.TCR.all = 0x4000;
    CpuTimer1Regs.TCR.all = 0x4000;
    CpuTimer2Regs.TCR.all = 0x4000;

    init_serialSCIA(&SerialA,115200);
    //    init_serialSCIC(&SerialC,115200);
    //    init_serialSCID(&SerialD,115200);

    //    To know which Registry needs EALLOW, this in in the condensed registries and then
    //    "11.16.2 ADC_REGS Registers" or "15.15.2 EPWM_REGS Registers"
    EALLOW;
```

```
265
266     //    TBCTL
267     //    Count up Mode
268     //    Free Soft emulation = Free Run
269     //    Do not load time time-base counter fro mthe time-base phase registry
270     //    Clock divide = 1
271     EPwm12Regs.TBCTL.bit.FREE_SOFT = 0b10;
272     EPwm12Regs.TBCTL.bit.CLKDIV = 0;
273     EPwm12Regs.TBCTL.bit.CTRMODE = 0;
274     EPwm12Regs.TBCTL.bit.PHSEN = 0;
275
276
277     //    TBCTR
278     //    Start Timer at zero
279     EPwm12Regs.TBCTR = 0;
280
281     //    TBPRD
282     //    Set period of the PWM to 5kHz (200 us)
283     //    Counter frequency is 50MHz
284     EPwm12Regs.TBPRD = 10000;
285
286     //    CMPA
287     //    Duty cycle at 50%, TBPRD / 2
288     EPwm12Regs.CMPA.bit.CMPA = 5000;
289
290     EPwm12Regs.AQCTLA.bit.CAU = 0b01;// PWM12A output pin is set low when CMPA is reached
291     EPwm12Regs.AQCTLA.bit.ZRO= 0b10; //the pin be set high when the TBCTR register is zero
292
293     EPwm12Regs.TBPHS.bit.TBPHS =0;
294
295     GpioCtrlRegs.GPAPUD.bit.GPIO22 = 1; // For EPW12A
296     EDIS;
297
298
299     // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
300     // which is connected to CPU-Timer 1, and CPU int 14, which is connected
301     // to CPU-Timer 2:  int 12 is for the SWI.
302     IER |= M_INT1;
303     IER |= M_INT8;  // SCIC SCID
304     IER |= M_INT9;  // SCIA
305     IER |= M_INT12;
306     IER |= M_INT13;
307     IER |= M_INT14;
308
309     // Enable TINT0 in the PIE: Group 1 interrupt 7
310     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
311     // Enable SWI in the PIE: Group 12 interrupt 9
312     PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
313
314     // Enable global Interrupts and higher priority real-time debug events
315     EINT;  // Enable Global interrupt INTM
316     ERTM;  // Enable Global realtime interrupt DBGM
317
318
319     // IDLE loop. Just sit and loop forever (optional):
320     while(1)
321     {
322         if (UARTPrint == 1 ) {
323             serial_printf(&SerialA,"Num Timer2:%ld Num SerialRX: %ld\r\n",CpuTimer2.InterruptCount,numRXA);
324             UARTPrint = 0;
325         }
326     }
327 }
328
329
330 // SWI_isr,  Using this interrupt as a Software started interrupt
331 __interrupt void SWI_isr(void) {
332
333     // These three lines of code allow SWI_isr, to be interrupted by other interrupt functions
334     // making it lower priority than all other Hardware interrupts.
335     PieCtrlRegs.PIEACK.all = PIEACK_GROUP12;
336     asm("       NOP");                  // Wait one cycle
337     EINT;                               // Clear INTM to enable interrupts
338
339
340
341     // Insert SWI ISR Code here.......
342
343
344     numSWIcalls++;
345
346     DINT;
347
348 }
349
350 // cpu_timer0_isr - CPU Timer0 ISR
351 __interrupt void cpu_timer0_isr(void)
352 {
353     CpuTimer0.InterruptCount++;
```

```
354
355     numTimer0calls++;
356
357     //    if ((numTimer0calls%50) == 0) {
358     //        PieCtrlRegs.PIEIFR12.bit.INTx9 = 1;  // Manually cause the interrupt for the SWI
359     //    }
360
361     if ((numTimer0calls%250) == 0) {
362 //        displayLEDletter(LEDdisplaynum);
363         LEDdisplaynum++;
364         if (LEDdisplaynum == 0xFFFF) {  // prevent roll over exception
365             LEDdisplaynum = 0;
366         }
367     }
368
369     // Blink LaunchPad Red LED
370     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
371
372     // Acknowledge this interrupt to receive more interrupts from group 1
373     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
374 }
375
376 // cpu_timer1_isr - CPU Timer1 ISR
377 __interrupt void cpu_timer1_isr(void)
378 {
379
380
381     CpuTimer1.InterruptCount++;
382 }
383
384 // cpu_timer2_isr CPU Timer2 ISR
385 __interrupt void cpu_timer2_isr(void)
386 {
387
388
389     // Blink LaunchPad Blue LED
390     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
391
392     CpuTimer2.InterruptCount++;
393
394     if ((CpuTimer2.InterruptCount % 50) == 0) {
395         UARTPrint = 1;
396     }
397 }
```